EV Navigation App — MVP & Build Plan

0. Reality check first

This is effectively **ABRP (A Better Route Planner) + PlugShare + Google Maps EV mode**, rebuilt India-first and multi-vehicle-class (2W/car/bus). That's the right comparison set to study before writing code — look at what ABRP, PlugShare, Tata Power's app, and Google Maps' native EV routing already do, and find the gap you can actually own (most likely: **India-specific data quality + 2-wheeler support**, which none of the big players do well). Don't try to out-build Google Maps' base mapping — build on top of it.

Given your background, the realistic path is: **hackathon-grade MVP in 6-8 weeks → pick ONE wedge (e.g. Bengaluru 2W/EV charger finder with battery-aware routing) → validate → expand.**

---

1. Layered architecture

You are **not** building a maps engine. You're building the battery/routing intelligence layer on top of Google Maps Platform or Mapbox.

---

2. Charger data — sources, and yes, open data exists

You don't need to scrape or negotiate 20 deals to launch.

Free/open sources (start here):

- **Open Charge Map (OCM)** — global open-data registry of charging locations, community-maintained since 2011, free public API (`GET /api/v3/poi`), returns location, connector types, power, and operator. Not real-time availability, but great for coverage/seeding. Data licensing is per-location (mostly CC-BY-SA-ish), so check attribution per POI, not blanket.
- **BEE (Bureau of Energy Efficiency) national database** — India's official public charging station registry (evyatra.beeindia.gov.in), growing fast under the **PM E-DRIVE scheme** (₹2,000 cr outlay, targeting ~72,000 new public charging points, prioritizing highways and high-EV-density cities). As of Aug 2025 there were ~29,300 public stations nationally, concentrated in Karnataka, Maharashtra, UP, Delhi, Tamil Nadu. Worth checking if BEE exposes a public API/export — if not, this is a candidate for a govt data RTI/partnership request later, not v1.
- **OpenStreetMap** — has many charger POIs tagged `amenity=charging_station`; free, but patchy in India.

Real-time availability + pricing (this is the hard part):

- Real-time status and ₹/kWh pricing lives with the network operators (Tata Power EZ Charge, Statiq, ChargeZone, Ather Grid, ChargeGrid, Jio-bp Pulse, etc.). Most don't have open public APIs — you'd need **OCPI (Open Charge Point Interface)**, the industry-standard protocol for exchanging real-time status/pricing between networks and apps like yours. This means direct partnerships with each network, which is a business-dev task, not a coding task, and it's the actual moat here — not the app UI.
- **v1 pragmatic approach:** launch with OCM + BEE static/semi-live data (location, connector type, rated power), let users self-report live availability (crowdsourced, PlugShare-style — "charger free/occupied/broken"), and treat real OCPI integrations as a Phase 2 partnership goal once you have users to show operators.

---

3. Vehicle telemetry (Bluetooth/OBD)

- **Cars (ICE-era OBD-II ports don't carry EV battery data reliably)** — most EVs expose battery %, charging rate, etc. only via **manufacturer-specific protocols**, not standard OBD-II PIDs. Realistic v1 options:
  - Use a **generic BLE OBD-II dongle** (ELM327-compatible) for basic telemetry (speed, GPS) where it's exposed.
  - For real EV-specific data (battery %, degradation, thermal state), you'd integrate manufacturer APIs where they exist (e.g. Tesla API, some apps use unofficial reverse-engineered endpoints for Tata/MG/Hyundai — legally grey, fragile, breaks on OTA updates).
  - **Simplest, most robust v1: let the user manually input/confirm battery % rather than depend on live vehicle telemetry.** Add BLE integration as a stretch goal per-brand later.
- **2-wheelers/EV buses** — even less standardized; some (Ather, Ola) have their own companion-app APIs (not public). Bus fleets usually run their own telematics (state

transport corporations) — that's an enterprise integration, not a consumer BLE pairing.
- **Realistic MVP telemetry stack:** phone GPS (speed, altitude via device sensors) + manual/periodic battery % input + optional BLE OBD dongle for cars where available. Don't block the MVP on live CAN-bus access — almost no consumer EV app has solved this cleanly.

---

4. Voice/chat assistant

- Don't chase "which AI has the best voice" — chase **latency + tool-calling reliability**, since the assistant's job is mostly: interpret intent → call your routing/charger functions → speak the result.
- Practical options: OpenAI's realtime voice API (GPT-4o-realtime-class models) or Claude/Gemini for the reasoning+tool-calling backend with a separate TTS/STT layer (e.g. ElevenLabs or Google/Azure speech) if you want more natural voice. Grok isn't meaningfully ahead here for a driving-assistant use case — pick based on API maturity for real-time audio + function calling, not brand.
- MVP scope for the assistant: **"find me a charger with 20+ km left," "how's my battery holding," "read out the next 3 stops," "what's traffic like ahead."** Keep it narrow and reliable rather than general-purpose chat — a flaky voice assistant while driving is worse than none.

---

5. Routing/battery model (your actual core IP)

This is the hardest technical piece and the real differentiator vs. plain Google Maps:

Battery consumption per road segment ≈ function of:

- Speed profile (highway vs city)
- Elevation gain/loss (climbs cost more than they're recovered on descents via regen)
- Vehicle weight/aero profile (varies drastically 2W vs bus)
- Temperature/weather (AC/heating load, cold-weather battery efficiency loss)
- Road surface class (your "clean highway, less degradation" idea — frame this as **smooth/well-maintained road preference to reduce mechanical + thermal stress**, since "battery degradation" isn't really a per-trip road effect, it's driving style + heat + fast-charging frequency over the vehicle's life. Reframe the feature honestly as: *prefer routes with less stop-start traffic, steep gradients, and potholed segments* — that's real and buildable.)

Start with a simple linear energy model (Wh/km baseline × terrain/speed/temperature multipliers), calibrate later with real trip data you collect. This is exactly the ABRP approach — you don't need ML for v1, a solid physics-lite model beats a black box you can't debug.

---

6. MVP structure — phased

Phase 0 (2-3 weeks) — Prove the wedge

- Single city (Bengaluru), single vehicle class (2-wheelers or cars, pick one)
- Static charger data from OCM + manual seeding of Bengaluru chargers
- Basic route with charger stops overlaid on Google Maps SDK
- Manual battery % input, simple linear consumption estimate
- No voice assistant yet, no BLE

Phase 1 (MVP proper, 6-8 weeks)

- Vehicle profile onboarding (battery kWh, charging speed, connector type, class)
- Default route mode = "efficient/smooth road" vs "fastest" toggle
- Charger-stop suggestion logic (30% threshold rule, as you specified)
- Trip logging (duration, avg consumption, cost)
- Basic chat assistant (text first, voice later — voice is 3-5x the engineering effort)
- POIs near chargers (coffee, restrooms) via Google Places API

Phase 2

- Voice assistant with tool-calling
- BLE OBD integration for supported vehicles
- Crowdsourced live charger status
- Expand to buses (fleet dashboard, different UX — this is closer to a B2B product than consumer app)

Phase 3

- OCPI partnerships with real networks for live pricing/availability
- Multi-city expansion
- ML-refined consumption model using your own collected trip data

---

7. Tech stack recommendation

Layer	Suggestion	Why
Mobile app	React Native or Flutter	Cross-platform, one codebase for car/2W/bus UI variants
Maps/routing base	Google Maps Platform (Directions, Roads, Places APIs) or Mapbox	Don't rebuild base mapping
Backend	Node.js/NestJS or Python/FastAPI	You already know NestJS (from your attendance system project)
DB	PostgreSQL + PostGIS	Geo-queries for "chargers near route"
Charger data ingestion	Scheduled jobs pulling OCM API + BEE data, normalized into your schema	
Telemetry	BLE via native modules (react-native-ble-plx)	
Assistant backend	OpenAI Realtime API or Claude + separate STT/TTS	
Trip analytics	Simple event logging → Postgres, dashboard later	You've already built this pattern in your GFLFGOA-GWO dashboard work
Hosting	Any (Vercel/Railway for backend, Supabase if you want managed Postgres+auth fast — you've used it before)	

8. Honest risks to flag now

- **Real-time charger availability is a business-dev problem, not an engineering one** — no amount of good code gets you OCPI data without operator partnerships.
- **EV telemetry access is fragmented per brand** — design the product to work well *without* live vehicle data (manual input) so you're not blocked.
- **Google already ships EV-aware routing** in Maps for supported vehicles — your differentiation has to be India charger data quality, 2-wheeler support, and the "relax while charging" experience layer, not the base routing.

9. 50 feature ideas (brainstormed, grouped)

Routing & battery

1. Battery-aware route (default) vs fastest-route toggle

2. Auto charger-stop insertion when range won't cover the trip
3. 30%-charge-triggers-search rule (as you specified)
4. Road-quality/smoothness preference weighting
5. Elevation-aware range estimate (climbs vs descents)
6. Weather-adjusted range (cold/heat/rain drag)
7. Regenerative braking credit in route scoring
8. Multi-stop trip planner with charge-time budgeting
9. "Arrive with X% buffer" safety margin setting
10. Alternate route comparison (time vs energy tradeoff shown side by side)
11. Live traffic-adjusted consumption re-estimate mid-trip
12. Convoy mode for fleet/bus routing (multiple vehicles, shared stops)
13. Night vs day route preference (some chargers unsafe/closed at night)
14. Toll-avoidance + energy-optimal combined routing

Charging experience 15. Charger connector-type filter (CCS2, Type 2, GB/T, Bharat AC/DC001, etc.) 16. Brand-preferred charger first, alternatives ranked after 17. Live price per kWh comparison across nearby stations 18. Charger occupancy prediction (historical pattern-based) 19. Reserve-a-slot (where operator API allows) 20. "What to do while charging" — cafes, restrooms, malls within walking distance 21. Estimated wait time at busy stations 22. Charger reliability score (crowdsourced uptime reports) 23. Fast vs slow charger toggle with time-cost shown 24. Battery swap station support (for 2W/3W — India-relevant, unlike most EV apps) 25. Multi-vehicle queue view for fleet operators at a depot

Vehicle & personalization 26. Vehicle profile setup wizard (battery size, charge curve, connector, class) 27. Charge-curve-aware time estimate (fast charging tapers near 80%) 28. Degradation-aware range estimate as battery ages (from trip history) 29. Vehicle-specific efficiency learning from actual trips 30. Support profiles for 2W, 3W, car, bus separately 31. Multiple vehicle profiles per user (household with 2 EVs)

Telemetry & live data 32. BLE pairing for supported vehicles (battery %, speed, temp) 33. Manual battery input fallback (always available) 34. Live altitude/gradient display 35. Average speed & efficiency live readout 36. Trip replay with battery graph over time

Assistant 37. Voice queries: "find charger," "how's my range," "read next stop" 38. Proactive voice alert: "charge is at 30%, no chargers in next 20km, rerouting" 39. Chat-based trip planning ("plan a trip to Mysuru with 2 stops") 40. Multilingual voice support (Kannada, Hindi, Tamil, etc. — relevant for India)

Trip history & analytics 41. Trip log: duration, distance, avg efficiency, cost 42. Battery health trend dashboard over months 43. Cost-per-km tracking vs fuel equivalent comparison 44. Driving-style efficiency score/tips 45. Exportable trip reports (useful for fleet/commercial users)

City & infrastructure (broader than trip routing) 46. City-level charger density heatmap (planning tool for urban EV adoption) 47. Public transport EV bus live tracking + charging

status (city ops use case) 48. Municipal dashboard: charger coverage gaps by ward/zone 49. Peak-hour grid load view (useful if you ever integrate with DISCOM data) 50. Community reporting: broken chargers, illegal ICE parking in EV bays

---

10. Suggested first move

Given your timeline and background, I'd scope Phase 0 as a **hackathon-style demo**: Bengaluru, 2-wheelers OR cars (not both), OCM data, manual battery input, static route + charger overlay, basic trip log. That's buildable in weeks with tools you already know (React Native/React, NestJS, Supabase/Postgres — all from your CareSync and attendance-system stack), and it's demoable for hackathons/M360-style applications without needing OCPI partnerships or BLE telemetry solved yet.